

Лабораторная работа по дисциплине «Интеллектуальные системы»

Выполнил: Миронов Егор Петрович, гр. 09-535

1. Описание моделируемой системы

В данной работе рассматривается поведение неигрового персонажа в роли охранника (NPC), управляемого компьютером, а не человеком в игре, обладающей стелс составляющей.

Для общего понимания предоставляю определения к ранее озвученным терминам:

- NPC (от англ. Non-Player Character) – это неигровой персонаж, управляемый компьютером, а не человеком;
- Стелс-игры (stealth, «невидимка») – это жанр, где основной целью является избегание обнаружения игрока противниками.

Основной задачей неигрового персонажа является обнаружение и преследование игрока, а также реагирование на подозрительную активность в зоне патрулирования. Поведение NPC является реактивным и зависит от текущего состояния окружающей среды, включая наличие или отсутствие игрока, уровень уверенности в его обнаружении, а также внутренние состояния охранника (усталость, тревога, режим поиска).

2. Определение модели Крипке

Модель поведения NPC-охранника задается как структура Крипке:

$$M = (S, I, R, L)$$

где:

- S – конечное множество состояний
- I – множество начальных состояний ($I \subseteq S$)
- R – отношение переходов между состояниями ($R \subseteq S \times S$)
- L – функция разметки состояний ($L: S \rightarrow 2^{AP}$)

Пусть $AP = \{p, c, s, a, r, i, w\}$ – множество атомарных высказываний, обозначения которых представлено в таблице 1:

Таблица 1 – Атомарные высказывания

Обозначение	Смысл
p	Замечено присутствие игрока (player detected)
c	Преследование игрока (chase)
s	Поиск игрока (search)
a	Состояние тревоги (alert)
r	Состояние отдыха (rest)
i	Исследование подозрительной местности (investigate)
w	Слабое подозрение игрока/игровой области (weak suspicion)

Конечное множество состояний S и множество начальных состояний I можно задать следующим образом:

$$S = \{S_0, S_1, S_2, S_3, S_4, S_5, S_6, S_7, S_8, S_9\}, I = \{S_0\}$$

Тогда разметку состояний модели можно зафиксировать так, как это показано в таблице 2:

Таблица 2 – Множество состояний и функция разметки

Состояние	$L(s)$	Описание
S_0	$\emptyset$	Патрулирование области
S_1	$\emptyset$	Ожидание
S_2	$\{p\}$	Обнаружено подозрение
S_3	$\{p, c\}$	Преследование игрока
S_4	$\{s\}$	Поиск игрока
S_5	$\emptyset$	Возвращение к патрулированию
S_6	$\{a, c\}$	Тревога
S_7	$\{r\}$	Отдых
S_8	$\{i, p\}$	Исследование конкретной области
S_9	$\{w, p\}$	Слабое подозрение конкретной области

Отношение переходов между состояниями системы демонстрирует таблица 3:

Таблица 3 – Переходы между состояниями

Из состояния	В состояние	Событие/условие перехода
S_0	S_1	NPC временно останавливается на посту
S_0	S_2	Обнаружено подозрительное движение
S_0	S_7	NPC завершил цикл патрулирования и уходит в режим отдыха
S_1	S_0	Возобновление патрулирования
S_1	S_2	Визуальный контакт с подозреваемым объектом
S_2	S_3	Подозреваемый объект = игрок
S_2	S_9	Подозреваемый объект != игрок
S_3	S_6	Игрок активно сопротивляется, переход в состояние тревоги
S_3	S_4	Потеря визуального контакта с игроком
S_4	S_5	Игрок не найден за время его поиска
S_5	S_0	Возврат на маршрут патрулирования
S_6	S_4	Ситуация обнаружения неясна, переход к поиску
S_7	S_0	Завершение отдыха
S_8	S_4	Объект не подтверждён как игрок
S_8	S_0	Ложное обнаружение
S_9	S_8	Усиление подозрения
S_9	S_4	Переход к активному поиску

3. CTL-формулы для верификации системы

Для верификации корректности протокола сформулированы следующие требования на языке CTL. Требования к системе продемонстрированы в таблице 4:

Таблица 4 – Требования к системе

Номер	Формула	Пояснение
1	$AG(p \rightarrow AF(c))$	Если игрок обнаружен, то во всех возможных путях в будущем обязательно начнётся преследование. Интерпретация: NPC не игнорирует игрока и любое обнаружение ведет к преследованию
2	$AG(a \rightarrow AF(c \vee s))$	Если активна тревога, то NPC обязательно либо начнёт преследование, либо перейдёт к поиску. Интерпретация: состояние тревоги приводит NPC в действие
3	$EF(a)$	Существует хотя бы один сценарий, где NPC входит в состояние тревоги. Интерпретация: состояние тревоги вполне достижимо.
4	$AG(c \rightarrow AF(\neg c))$	Если NPC начал преследование, то он не может находиться в нём бесконечно. Интерпретация: игрок может скрыться и состояние преследования завершится.

4. Алгоритм маркировки состояний

Алгоритм маркировки состояний вычисляет для каждой подформулы φ CTL-формулы множество состояний $Sf = \{s \in S \mid M, s \models \varphi\}$, в которых φ истинна.

Ключевые правила вычисления:

- $Sf[p] = \{s \in S \mid p \in L(s)\}$ – атомарное высказывание;
- $Sf[\neg\varphi] = S \setminus Sf[\varphi]$ – отрицание;
- $Sf[\varphi_1 \wedge \varphi_2] = Sf[\varphi_1] \cap Sf[\varphi_2]$ – конъюнкция;
- $Sf[\varphi_1 \vee \varphi_2] = Sf[\varphi_1] \cup Sf[\varphi_2]$ – дизъюнкция;
- $Sf[EX\varphi] = \{s \mid \exists s': (s, s') \in R \wedge s' \in Sf[\varphi]\}$ – квантор EX;
- $Sf[EF\varphi]$ – означает достижимость состояния, где φ истинно;
- $Sf[EG\varphi]$ – существует путь, где φ истинно во всех состояниях;
- $Sf[E(\varphi_1 \cup \varphi_2)]$ – существует путь, где φ_1 истинно до тех пор, пока

не станет истинным φ_2 ;

- $Sf[AG\varphi] = Sf[\neg EF\neg\varphi]$

$$- Sf[AF\varphi] = Sf[\neg EG\neg\varphi]$$

4.1. Проверка CTL-формулы 1

Формула 1: $AG(p \rightarrow AF(c))$ – если в состоянии обнаружен игрок (p), то на всех путях система обязательно в будущем придёт к состоянию преследования (c).

Шаг 1. Преобразование формулы:

$$AG(p \rightarrow AF(c)) = \neg EF(\neg(p \rightarrow AF(c)))$$

Шаг 2. Базовые множества:

$$f_1 = Sf(c) = \{S_3, S_6\}$$

$$f_2 = EU(True, f_1) = \{S_0, S_1, S_2, S_3, S_4, S_5, S_6, S_8, S_9\}$$

Шаг 3. Импликация:

f_3 – где нарушается импликация, а она ложна только если

$$- p = True;$$

$$- AF(c) = False.$$

$f_3 = Sf(p) \setminus f_2$, но $Sf(p) = \{S_2, S_3, S_8, S_9\} \subseteq f_2$, следовательно $f_3 = \emptyset$

Шаг 4. Проверка EF:

$$f_4 = EF(f_3) = EU(True, f_3) = \emptyset$$

Шаг 5. Итог:

$$AG(p \rightarrow AF(c)) = \neg f_4 = \neg \emptyset = S$$

Пришли к истине, формула выполняется во всех состояниях

4.2. Проверка CTL-формулы 2

Формула 2: $AG(a \rightarrow AF(c \vee s))$ – если активна тревога (a), то во всех путях система обязательно придёт либо к преследованию (c), либо к поиску (s).

Шаг 1. Преобразование формулы:

$$AF(c \vee s) = EU(True, (c \vee s))$$

Шаг 2. Базовые множества:

$$f_1 = Sf(c) = \{S_3, S_6\}$$

$$f_2 = Sf(s) = \{S_4\}$$

$$f_3 = Sf(c \vee s) = \{S_3, S_4, S_6\}$$

$$f_4 = EU(True, f_3) = \{S_0, S_1, S_2, S_3, S_4, S_5, S_6, S_8, S_9\}$$

$$f_5 = Sf(a) = \{S_6\}$$

Шаг 3. Импликация:

Импликация нарушается только если:

- $a = True$
- $AF(c \vee s) = False$

$$f_6 = f_5 \setminus f_4 = \emptyset, \text{ т.к. } S_6 \in f_4$$

Шаг 4. Проверка EF:

$$f_7 = EF(f_6) = EU(True, f_6) = \emptyset, \text{ т.к. } f_6 = \emptyset$$

Шаг 5. Итог:

$$AG(a \rightarrow AF(c \vee s)) = \neg f_7 = \neg \emptyset = S$$

Пришли к истине, значит система всегда корректно реагирует на тревогу.

4.3. Проверка CTL-формулы 3

Формула 3: $EF(a)$ – существует хотя бы один путь, на котором NPC когда-либо входит в состояние тревоги (a).

Шаг 1. Базовые множества:

$$f_1 = Sf(a) = \{S_6\}$$

Шаг 2. Вычисление $EF(a)$:

$$EF(a) = EU(True, a)$$

$$f_2 = \{S_6\}$$

Поиск предшественников:

- $S_3 \rightarrow S_6$
- $S_6 \rightarrow S_6$
- $S_2 \rightarrow S_3 \rightarrow S_6$
- $S_0 \rightarrow S_2 \rightarrow S_3 \rightarrow S_6$
- $S_1 \rightarrow S_2 \rightarrow S_3 \rightarrow S_6$

- $S_8 \rightarrow S_4 \rightarrow S_3 \rightarrow S_6$
- $S_9 \rightarrow S_8 \rightarrow S_4 \rightarrow S_3 \rightarrow S_6$

Фиксированная точка: $f_3 = \{S_0, S_1, S_2, S_3, S_4, S_6, S_8, S_9\}$

Шаг 3. Итог:

$$EF(a) = f_3$$

$$EF(a) = True$$

Состояние тревоги S_6 достижимо, существует несколько путей, ведущих в него, система может перейти в режим тревоги при определённых условиях.

4.4. Проверка CTL-формулы 4

Формула 4: $AG(c \rightarrow AF(\neg c))$ – если NPC начал преследование игрока (c), то он обязательно в будущем выйдет из состояния преследования

Шаг 1. Базовые множества:

$$f_1 = Sf(c) = \{S_3, S_6\}$$

$$f_2 = Sf(\neg c) = S \setminus f_1 = \{S_0, S_1, S_2, S_4, S_5, S_7, S_8, S_9\}$$

Шаг 2. Вычисление $AF(\neg c)$:

$$f_3 = EU(True, \neg c) = \{S_0, S_1, S_2, S_4, S_5, S_7, S_8, S_9\}$$

Проверяем, можно ли из состояний c перейти в $\neg c$:

- $S_3 \rightarrow S_4$
- $S_6 \rightarrow S_4$

значит оба состояния c имеют выход, фиксированная точка $f_3 = S$

Шаг 3. Проверка импликации:

Импликация нарушается только если:

- $c = True$
- $AF(\neg c) = False$

$$f_4 = f_1 \setminus f_3 = \emptyset, \text{ т.к. } f_1 \subseteq f_3$$

Шаг 4. Проверка EF:

$$f_5 = EF(f_4) = EU(True, f_4) = \emptyset, \text{ т.к. } f_4 = \emptyset$$

Шаг 5. Итог:

$$AG(c \rightarrow AF(\neg c)) = \neg f_5 = \neg \emptyset = S$$

Пришли к истине, значит NPC не застревает в преследовании.